

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

ОТЧЁТ ПО УЧЕБНОЙ ПРАКТИКЕ

Тема: Генетические алгоритмы

Студент гр. 4382

А.Д. Смирнов

Студент гр. 4382

В.А. Росицкий

Студент гр. 4383

Г.А. Гамага

Руководитель

Т.Р. Жангиров

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: А.Д. Смирнов

Группа: 4382

Студент: В.А. Росицкий

Группа: 4382

Студент: Г.А. Гамага

Группа: 4383

Тема практики: Генетические алгоритмы

Задание на практику:

Дано N точек в двумерном евклидовом пространстве. Необходимо найти координаты левого нижнего угла и длины сторон для M (задается пользователем) квадратов такие, чтобы квадраты суммарно содержали максимальное количество точек внутри и не пересекались между собой.

Программа должна иметь GUI с пошаговой визуализацией, настройкой параметров алгоритма пользователем. Должен присутствовать график изменения функции качества решения от номера популяции, который дополняется с каждым шагом алгоритма.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 17.07.2026

Дата защиты отчета: 17.07.2026

Студент гр.4382

А.Д. Смирнов

Студент гр.4382

В.А. Росицкий

Студент гр. 4383

Г.А. Гамага

Руководитель

Т.Р. Жангиров

АННОТАЦИЯ

В данном проекте реализован генетический алгоритм для решения задачи покрытия точек непересекающимися квадратами. Программы написаны на языке *Python* с использованием библиотек *PyQt5* для создания графического интерфейса и *Matplotlib* для отрисовки графиков. Проект разрабатывался с использованием системы контроля версий *git* и платформы *GitHub*.

SUMMARY

This project implements a genetic algorithm to solve the problem of covering points with disjoint squares. The programs are written in Python using the PyQt5 library to create a GUI and the Matplotlib library to draw graphs. The project was developed using the git version control system and the GitHub platform.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1 ПОСТАНОВКА ЗАДАЧИ	6
1.1 Предметная область	6
1.2 Задачи практики	6
1.3 Третий раздел первой главы	6
1.3.1 Первый подраздел	6
2 РЕЗУЛЬТАТЫ РАБОТЫ	7
2.1. Формулировка задачи 1	7
2.2. Формулировка задачи 2	7
2.3. Формулировка задачи 3	7
3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ	8
4 ОТЗЫВ РУКОВОДИТЕЛЯ	9
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11
ПРИЛОЖЕНИЕ А	12

ВВЕДЕНИЕ

Предметной областью учебной практики являются генетические алгоритмы и их применение для решения задач оптимизации. Генетические алгоритмы относятся к эвристическим методам поиска решений и используются в случаях, когда полный перебор вариантов является слишком трудоемким или практически невозможным. В рамках данной работы генетический алгоритм применяется для решения задачи покрытия множества точек на плоскости набором непересекающихся квадратов.

Целью практики является разработка программы с графическим интерфейсом для поиска приближенного решения задачи покрытия точек непересекающимися квадратами. Программа должна позволять задавать входные данные, настраивать параметры генетического алгоритма, визуализировать текущие решения и отображать изменение функции качества по поколениям.

Для достижения цели были поставлены следующие задачи: изучить основные принципы работы генетических алгоритмов, разработать структуру графического интерфейса, реализовать ввод исходных точек, подготовить визуализацию точек и квадратов, создать реализацию генетического алгоритма и реализации операторов отбора, скрещивания, мутации и вычисления функции качества.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Генетические алгоритмы применяются для решения оптимизационных задач, в которых требуется найти достаточно хорошее решение среди большого числа возможных вариантов. Такие методы используются при планировании, размещении объектов, подборе параметров, маршрутизации и других задачах, где невозможно эффективно проверить все возможные комбинации.

В данной работе рассматривается задача покрытия точек квадратами. На вход подаётся набор точек на плоскости и количество квадратов, которое задаётся пользователем. Требуется подобрать координаты левых нижних углов квадратов и длины их сторон так, чтобы суммарно покрыть максимальное количество точек. При этом квадраты не должны пересекаться между собой. Задача является оптимизационной, так как необходимо максимизировать значение функции качества. В качестве функции качества используется количество покрытых точек с учётом штрафа за пересечение квадратов.

1.2 Задачи практики

В рамках учебной практики необходимо выполнить следующие задачи:

1. Сформировать структуру проекта и выбрать средства разработки.
2. Разработать прототип графического интерфейса.
3. Реализовать ввод исходных данных.
4. Реализовать визуализацию точек и квадратов.
5. Добавить настройку параметров генетического алгоритма через GUI.
6. Подготовить структуру класса генетического алгоритма.
7. Реализовать генетический алгоритм.
8. Реализовать загрузку и выгрузку данных
9. Реализовать пошаговую визуализацию работы алгоритма.

10. Реализовать график изменения функции качества.

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Формирование структуры проекта

На начальном этапе был создан репозиторий проекта *genetic-optimization-gui*. Для реализации программы был выбран язык Python 3. Проект был разделён на несколько файлов. Основной файл *MainWindow.py* содержит класс главного окна приложения. Файл *VisualWidget.py* содержит класс, отвечающий за отображение точек и квадратов. Файл *GeneticAlgorithm.py* предназначен для хранения параметров и дальнейшей реализации генетического алгоритма. Файл *DataUtils.py* используется для вспомогательных операций с входными данными. Также имеется файл *Constants.py* для хранения константных значений. Такое разделение позволяет не перегружать один файл всей логикой программы. Код интерфейса, визуализации, алгоритма и работы с данными постепенно выносятся в отдельные части проекта. Общая структура классов разрабатываемой программы представлена на рисунке 1.

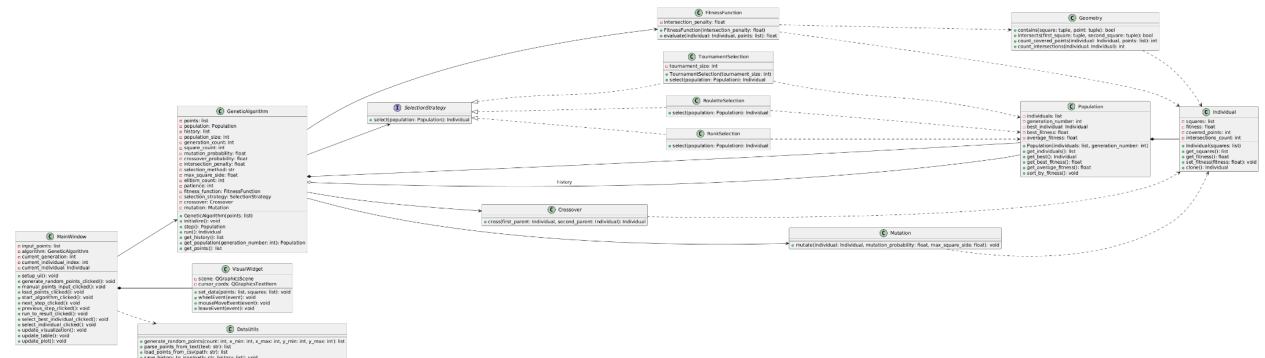


Рисунок 1— UML-диаграмма классов проекта

На UML-диаграмме показана целевая архитектура программы. Класс *MainWindow* отвечает за графический интерфейс, обработку действий пользователя и связь с генетическим алгоритмом. В нём хранятся входные точки, текущий выбранный шаг, индекс текущего индивида и ссылка на объект генетического алгоритма. Класс *VisualWidget* используется для

визуализации точек и квадратов. Класс *DataUtils* содержит вспомогательные методы для генерации, ручного ввода, загрузки и сохранения данных.

Алгоритмическая часть представлена классом *GeneticAlgorithm*, который управляет процессом поиска, хранит историю поколений и параметры генетического алгоритма, заданные пользователем. Возможное решение задачи описывается классом *Individual*, который содержит набор объектов *Square*. Несколько решений одного поколения объединяются в класс *Population*. Для вычислительной части выделены классы *Geometry* и *FitnessFunction*. Класс *Geometry* выполняет геометрические проверки: попадание точки в квадрат, пересечение квадратов и подсчёт покрытых точек. Класс *FitnessFunction* вычисляет значение функции качества решения.

Генетические операторы вынесены в отдельные классы. За отбор родителей отвечает интерфейс *SelectionStrategy* и его реализации *TournamentSelection*, *RouletteSelection* и *RankSelection*. Скрещивание выполняется классом *Crossover*, мутация – классом *Mutation*. Для хранения результатов по поколениям используется список популяций внутри класса *GeneticAlgorithm*. Каждая популяция содержит номер поколения, список особей, лучшую особь, лучшее значение функции качества и среднее значение функции качества.

2.2. Разработка прототипа графического интерфейса

Был разработан прототип графического интерфейса приложения с использованием библиотеки *PyQt5*. Главное окно реализовано в классе *MainWindow*. Интерфейс был разделён на три основные области. В левой части окна расположены элементы для ввода исходных данных и настройки параметров алгоритма. В центральной части находится область визуализации, предназначенная для отображения точек и квадратов. В правой части размещён график функции качества, который будет использоваться при пошаговой работе генетического алгоритма. В интерфейсе были подготовлены кнопки для генерации точек, загрузки данных, ручного ввода точек, запуска алгоритма, перехода по шагам и выбора текущего индивида.

Часть кнопок на текущем этапе уже получила обработчики, а часть оставлена для дальнейшей реализации. В дальнейших пунктах будет описана доработка этого прототипа.

2.3. Реализация ввода исходных точек

Были реализованы три способа задания исходных точек: случайная генерация, загрузка из файла и ручной ввод. Для хранения точек, введенных пользователем до запуска алгоритма, в классе *MainWindow* используется поле *input_points*. Это позволяет сначала подготовить входные данные в интерфейсе, а затем передать в объект класса *GeneticAlgorithm* при запуске алгоритма. Случайная генерация точек вынесена во вспомогательный класс *DataUtils*. После генерации точки сохраняются в *input_points*, обновляется информация о количестве точек и вызывается перерисовка области визуализации. С помощью ручного ввода пользователь может ввести координаты самостоятельно, после чего программа преобразует введенный текст в список точек и отобразит их в окне визуализации. Это удобно для проверки работы программы на небольших тестовых примерах. Также можно загрузить исходные точки из файла JSON или TXT, что позволяет удобно работать с какими-то конкретными точками, не вводя их каждый раз вручную.

2.4. Реализация визуализации точек и квадратов

Для отображения геометрических объектов был реализован класс *VisualWidget*. Он отвечает за отрисовку точек и квадратов в центральной части окна. Метод *set_data()* принимает список точек и список квадратов, очищает текущую сцену и заново отображает переданные объекты. Точки отображаются в виде небольших окружностей, а квадраты – в виде прямоугольников с одинаковой шириной и высотой. Также в области визуализации реализованы масштабирование с помощью колеса мыши, перемещение сцены и отображение координат курсора. Это позволяет удобнее анализировать расположение объектов на плоскости.

2.5. Настройка параметров генетического алгоритма через графический интерфейс

В левой панели главного окна приложения реализован блок элементов управления, позволяющий пользователю гибко задавать все основные параметры генетического алгоритма перед его запуском. Для ввода целочисленных параметров – размера популяции, числа поколений и количества квадратов – используются компоненты *QSpinBox* из библиотеки *PyQt5*. Для вещественных параметров – вероятности мутации, вероятности скрещивания, штрафа за пересечение квадратов и штрафа за пустые квадраты – применяются *QDoubleSpinBox* из библиотеки *PyQt5*. Все параметры ограничены константами, а также имеют определённое начальное значение, которое также задано константами. Константы находятся в файле *Constants.py*. Метод отбора родителей выбирается из выпадающего списка *QComboBox*, содержащего три варианта: «Турнирный отбор», «Рулетка» и «Ранжированный отбор». Все параметры считываются в момент нажатия кнопки «Запустить алгоритм» и передаются в конструктор класса *GeneticAlgorithm*, который инициализирует процесс поиска с учётом заданных настроек. Таким образом, пользователь может проводить серию экспериментов, изменяя параметры и наблюдая за их влиянием на эффективность и скорость сходимости алгоритма.

2.6. План реализации генетического алгоритма.

Для решения задачи покрытия точек квадратами был выбран генетический алгоритм, так как задача является оптимизационной и требует подбора большого количества вещественных параметров. Входными данными алгоритма являются список точек на плоскости, количество квадратов, размер популяции, число поколений, вероятность мутации, вероятность скрещивания, штраф за пересечение квадратов и метод отбора родителей. Точки хранятся в виде пар координат (x, y), без создания отдельного класса, поскольку точка в данной задаче не имеет собственного поведения и используется только в геометрических вычислениях.

Одна особь генетического алгоритма представляет собой один вариант расположения квадратов. Если пользователь задает M квадратов, то особь содержит набор $I = [s_1, s_2, \dots, s_M]$, где каждый квадрат задается тройкой параметров $s_j = (x_j, y_j, a_j)$. Здесь x_j и y_j - координаты левого нижнего угла квадрата, а a_j - длина его стороны. Таким образом, хромосома особи представляет собой набор вещественных значений, описывающих координаты и размеры всех квадратов.

Качество особи оценивается с помощью функции приспособленности $F(I) = C(I) - aK(I)$, где $C(I)$ - количество уникальных точек, покрытых квадратами данной особи, $K(I)$ - количество пар пересекающихся квадратов, а a - коэффициент штрафа за пересечение. Такая функция качества позволяет одновременно учитывать основную цель задачи - покрытие максимального количества точек - и ограничение на непересечение квадратов. Чем больше значение функции приспособленности, тем более удачным считается решение.

Работа генетического алгоритма планируется следующим образом: сначала создаётся начальная популяция случайных особей, затем для каждой особи вычисляется функция приспособленности, после чего выполняются отбор родителей, скрещивание, мутация и формирование нового поколения. В интерфейсе предусмотрен выбор метода отбора родителей. Основным вариантом планируется турнирный отбор, так как он устойчиво работает даже при отрицательных значениях функции качества, которые могут возникать из-за штрафов. Также в архитектуре предусмотрены рулеточный и ранговый отбор. Для сохранения лучших решений будет использоваться элитизм, а история поколений будет храниться в виде списка поколений внутри класса `GeneticAlgorithm`. Это позволит реализовать пошаговую визуализацию, построение графика изменения функции качества и возврат к предыдущим поколениям.

2.7. Реализация генетического алгоритма.

Для решения задачи покрытия точек квадратами была реализована алгоритмическая часть программы. Основной класс *GeneticAlgorithm* отвечает за хранение входных точек, параметров алгоритма, текущей популяции и истории поколений. При запуске алгоритма по исходному набору точек определяются границы рабочей области, после чего создаётся начальная популяция случайных решений. Каждое решение представлено объектом класса *Individual* и содержит набор квадратов. Один квадрат задаётся тройкой параметров (x, y, w) , где x и y - координаты левого нижнего угла, а w - длина стороны. Таким образом, хромосома индивида представляет собой список квадратов, которые являются возможным вариантом покрытия точек.

Для оценки качества решений была реализована функция приспособленности. Она учитывает количество покрытых точек, количество пересечений между квадратами и число пустых квадратов. За каждую покрытую точку значение функции качества увеличивается, а за пересечения, пустые квадраты вводятся штрафы. Такой подход позволяет алгоритму искать решения, которые покрывают как можно больше точек, но при этом не используют слишком большие или пересекающиеся квадраты. Значение *fitness* вычисляется для каждого индивида популяции и затем используется при выборе родителей для следующего поколения.

Переход к следующему поколению выполняется с помощью стандартных операторов генетического алгоритма. Для выбора родителей реализованы турнирный и ранговый отбор. Основным вариантом используется турнирный отбор: из популяции случайно выбирается несколько особей, после чего в качестве родителя берётся особь с наибольшим значением *fitness*. После отбора выполняется двухточечное скрещивание, при котором хромосомы двух родителей разбиваются на части и комбинируются между потомками. Затем к полученной популяции применяется мутация: отдельные квадраты могут случайно изменить своё

положение или размер стороны. После этого для новой популяции снова вычисляется функция качества, а сама популяция добавляется в историю алгоритма.

История поколений хранится внутри объекта *GeneticAlgorithm* в виде списка популяций. Это позволяет использовать результаты работы алгоритма в графическом интерфейсе: пользователь может переходить между поколениями, выбирать отдельного индивида, просматривать его хромосомы и сравнивать текущее решение с лучшим решением поколения. Также на основе истории поколений строится график изменения функции качества, где отображаются лучшее и среднее значения *fitness* для каждого поколения. Таким образом, реализованная алгоритмическая часть связана с интерфейсом и обеспечивает пошаговую визуализацию процесса поиска решения.

2.8. Реализация загрузки и выгрузки данных

Для расширения способов работы с исходными данными была реализована загрузка и сохранение информации через файлы форматов JSON и TXT. Эта часть вынесена во вспомогательный класс *DataUtils*, так как работа с файлами не относится напрямую к логике графического интерфейса. В классе *MainWindow* были добавлены обработчики кнопок загрузки точек, сохранения точек и сохранения популяций. Такой подход позволяет пользователю не только вводить точки вручную или генерировать их случайно, но и использовать заранее подготовленные наборы данных.

Загрузка точек выполняется из файлов JSON или TXT. В JSON-файле точки задаются в виде списка координат, а в TXT-файле каждая строка содержит одну точку. После загрузки координаты сохраняются в поле *input_points*, обновляется информация о количестве точек и вызывается перерисовка области визуализации. При загрузке нового набора точек состояние ранее запущенного алгоритма сбрасывается, так как старая история поколений уже не соответствует новым входным данным.

Также была реализована выгрузка текущего набора точек в файл. Это позволяет сохранить точки, полученные случайной генерацией или ручным вводом, и использовать их повторно при следующих запусках программы. Отдельно была добавлена возможность сохранения истории популяций после запуска генетического алгоритма. В файл записываются исходные точки, номера поколений, лучшее и среднее значение функции качества, а также параметры особей: значение *fitness* и набор квадратов с координатами левого нижнего угла и длиной стороны. Для предотвращения ошибок были добавлены проверки: программа сообщает пользователю о некорректном формате файла, отсутствии точек или попытке сохранить популяции до запуска алгоритма.

2.9. Пошаговая визуализация работы алгоритма

Для наглядного анализа процесса оптимизации в программе реализован механизм пошаговой визуализации. После запуска алгоритма вычисляется только первое поколение популяции, после чего пользователь может последовательно просматривать результаты работы алгоритма на каждом шаге.

Внутри класса *GeneticAlgorithm* все сгенерированные популяции сохраняются в списке *history*. При инициализации алгоритма в список добавляется начальная популяция. При каждом вызове метода *step()* вычисляется следующее поколение и добавляется в конец списка. На правой панели интерфейса расположены кнопки:

- «Предыдущий шаг» – программа переходит к прошлому шагу, если сейчас рассматривается не первое поколение, и перерисовывает визуализацию для предыдущего поколения.
- «Следующий шаг» – если текущий шаг равен максимальному вычисленному, то выполняется вычисление нового поколения, его добавление в таблицу и смена отображения. Если же текущий шаг

меньше максимального вычисленного, то просто сменяется отображение на текущее рассматриваемое поколение.

- «Перейти к результату» – запускает полный цикл работы алгоритма, вычисляя все поколения до заданного числа. После этого история пополняется, таблица заполняется всеми строками, и отображается последнее поколение.

При переходе на новое поколение виджет *VisualWidget* отображает исходные точки и квадраты, соответствующие выбранному индивиду из этого поколения. При переходе выбирается индивидуум популяции с максимальным значением функции приспособленности. Пользователь может рассмотреть остальных индивидов с помощью счётчика «Индивид». Кнопка «Выбрать лучшего» позволяет быстро вернуться к лучшему решению в текущей популяции. Кнопка «Посмотреть гены» открывает диалоговое окно, в котором для выбранного индивида выводятся координаты левого нижнего угла и длина стороны для каждого квадрата в виде таблицы.

В левой нижней части окна размещена таблица, в которой для каждого вычисленного поколения отображаются его номер, лучшее значение fitness и среднее значение fitness по популяции. При клике на любую строку таблицы происходит переход к соответствующему поколению. Также был реализован график функции качества, который также пошагово изменяется, но он будет описан более подробно в следующем пункте.

2.10. График изменения функции качества

Для отслеживания динамики работы алгоритма в правой части главного окна реализован график, построенный с использованием библиотеки Matplotlib. График имеет панель навигации, которая позволяет пользователю масштабировать, перемещать и сохранять график. На графике отображаются две кривые: Лучший результат – максимальное значение функции приспособленности среди всех особей в поколении (красная линия); средний

результат – среднее арифметическое fitness по всем особям поколения (синяя линия). Обе кривые строятся по оси абсцисс – номер поколения. При инициализации алгоритма график содержит только первое поколение. При каждом переходе на следующий шаг (кнопка «Следующий шаг») или при полном выполнении алгоритма («Перейти к результату») вызывается метод *draw_fitness_plot()*, который полностью перестраивает график на основе всей накопленной истории поколений, хранящейся в *algorithm.history*. Новые точки добавляются на график, оси автоматически подстраиваются под диапазон значений.

Таким образом, пользователь может наблюдать за сходимостью генетического алгоритма. Возможность интерактивного масштабирования позволяет детально рассмотреть поведение кривых на любом участке процесса.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

В ходе выполнения учебной практики используются следующие технологии:

1. Python 3 - основной язык программирования проекта. Используется для реализации графического интерфейса, вспомогательных функций и генетического алгоритма.
2. PyQt5 - библиотека для создания графического интерфейса. Используется для построения главного окна, кнопок, полей ввода, таблицы, выпадающих списков и области визуализации.
3. Matplotlib - библиотека для построения графиков. Используется для отображения графика изменения функции качества по поколениям.
4. Git - система контроля версий. Используется для фиксации изменений, работы по веткам и отслеживания этапов разработки.
5. GitHub - платформа для хранения удалённого репозитория проекта и совместной работы над кодом.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

Если прохождение практики было по кафедральным темам достаточно добавить формулировку:

По результатам работы учебная практика оценена на <ваша_оценка>.

Если прохождение практики было по своей теме, то необходимо добавить скан отзыва руководителя с оценкой и подписью, скан должен быть в высоком качестве на всю страницу.

Для прохождения автоматической проверки для скана необходима подпись и ссылка в тексте, т.к. он считается рисунком.

Отзыв руководителя с оценкой (см. рис. X).

Отзыв
о прохождении учебной практики

<ФИО студента>, студент(ка) группы <номер группы>, второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил(а) учебную практику по теме “<тема практики>” с <дата начала> по <дата окончания>.

В течение практики обучающийся(ая) <ФИО студента> выполнял <задание на практику>.

В результате практики обучающийся(ая) <результат практики>. Получаемую работу обучающийся(ая) <ФИО студента> выполнял <характеристика личных качеств практиканта>.

По результатам работы <ФИО студента> учебной практику оцениваю на <оценка: Отлично, Хорошо, Удовлетворительно, Неудовлетворительно>.

Подпись руководителя практики:

<Должность><дата> <подпись>/<ФИО>

Рисунок X — Отзыв руководителя

ЗАКЛЮЧЕНИЕ

Заключение содержит краткое описание результатов по каждой задаче, обозначенной в разделе «Постановка задачи», результаты работы в целом, согласно обозначенной цели и пр.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

ПРИЛОЖЕНИЕ А